

React.JS_Quick_Revision_Short_Note

<https://gemini.google.com/app/4602c44f8fd0820f>

User prompt: Solve All The Question and give me All The Question with an Answer Do Not Skip Any Question Give me All Question with an Answer Solve In All This Question and Explain in Detailed in simple with an Example in Exam and Technical Interview Point of view in Real Wolrd Programming Example # React.js Technical Interview Questions (1–100) #### Basic → Intermediate → Advanced | Real-World Focused --- ## SECTION 1: BASIC LEVEL (Q1–Q35) *Core concepts every React developer must know* #### Fundamentals 1. What is React? Why is it called a "library" and not a "framework"? 2. What is the Virtual DOM? Why does React use it instead of directly manipulating the real DOM? 3. How does the Virtual DOM improve performance? Explain the diffing/reconciliation process at a high level. 4. What is JSX? Why isn't it valid JavaScript on its own, and how does it get converted? 5. Can you write React without JSX? Give an example using `React.createElement`. 6. What is a React component? What is the difference between a functional and a class component? 7. What are props? Give a real-world example (e.g., passing user data to a `ProfileCard` component). 8. Are props mutable or immutable? What happens if a child tries to modify its own props? 9. What is state in React? How is it different from props? 10. What is the difference between controlled and uncontrolled components? Give a form example. #### Rendering & Lifecycle 11. What triggers a component to re-render in React? 12. What are keys in React lists, and why does React warn you when they're missing? 13. Why shouldn't you use array index as a `key` when the list can reorder or change? What real bug can this cause? 14. What is component composition? Give a real-world example (e.g., a `Layout` component wrapping `Header`, `Sidebar`, `Content`). 15. What is conditional rendering? Show a couple of common patterns (ternary, `&&`, early return). #### Hooks Basics 16. What are React Hooks? Why were they introduced (what problem did they solve)? 17. What is `useState`? Give a real-world example (e.g., toggling a modal open/closed). 18. What is `useEffect`? What real-world tasks would you put inside it (e.g., fetching data, subscribing to events)? 19. What is the dependency array in `useEffect`? What happens if you omit it vs pass an empty array `[]`? 20. What is the cleanup function in `useEffect`, and when would you need one (e.g., clearing an interval or unsubscribing)? #### Events & Forms 21. How does event handling in React differ from vanilla JavaScript (SyntheticEvent)? 22. How do you handle a form input's value with `useState`? What is a "controlled input"? 23. How do you handle multiple form fields with a single state object instead of many `useState` calls? 24. What is `preventDefault()` used for in a React form submit handler, and why is it usually necessary? 25. How would you validate a login form in React before submitting to an API? #### Styling & Basics (continued) 26. What are the different ways to style a React component (CSS files, CSS Modules, inline styles, styled-components)? 27. What is the `className` prop, and why isn't it called `class` like in HTML? 28. What is `React.Fragment` (`<>...</>`) used for, and why avoid unnecessary wrapper `

`'s? 29. What is the `children` prop? Give a real-world example (e.g., a reusable `Card` or `Modal` wrapper component). 30. What is prop drilling? Give a real-world scenario where it becomes a problem. #### Basic Project Structure 31. What is `create-react-app` / Vite, and what's the difference between them for starting a new project? 32. What is the role of the `public/index.html` file and the root `div` in a React app? 33. What is `ReactDOM.createRoot()` (React 18) used for? 34. How do you pass a function from a parent to a child component to handle a child-triggered event (e.g., a button click)? 35. What is the difference between a "presentational" and a "container" component in real-world app structuring? --- ## SECTION 2: INTERMEDIATE LEVEL (Q36–Q70) *Real-world architecture, performance, common interview traps* #### Hooks Deep Dive 36. What is `useContext`, and how does it help avoid prop drilling? Give a real-world example (e.g., theme or auth context). 37. What is `useRef`? Give two real-world use cases (DOM access, and persisting a mutable value without re-render). 38. What is the difference between `useRef` and `useState` in terms of triggering re-renders? 39. What is `useReducer`? When would you prefer it over `useState` (e.g., complex form

or shopping cart state)? 40. What are custom hooks? Write/describe a real-world custom hook (e.g., `useFetch`, `useDebounce`, `useLocalStorage`). 41. What rules must you follow when using Hooks (Rules of Hooks), and why do they exist? 42. What is `useMemo`, and when should you actually use it? Give a real-world case (e.g., expensive filtering/sorting of a large list). 43. What is `useCallback`, and how does it differ from `useMemo`? 44. Why can overusing `useMemo`/`useCallback` sometimes hurt performance instead of helping? 45. What is the difference between `useLayoutEffect` and `useEffect`? Give a real-world case where `useLayoutEffect` is necessary (e.g., measuring DOM before paint). #### Performance Optimization 46. What is `React.memo`, and when would you use it? Give a real-world example (e.g., a large list item component). 47. Why doesn't `React.memo` help if you're passing a new function or object as a prop on every render? 48. What is code-splitting in React? How does `React.lazy` and `Suspense` help with real-world bundle size issues? 49. How would you optimize the rendering of a list with thousands of items (virtualization, e.g., `react-window`)? 50. What causes unnecessary re-renders in a React app, and how would you debug them (React DevTools Profiler)? #### State Management 51. What is "lifting state up"? Give a real-world example (e.g., two sibling components needing shared filter state). 52. When would you reach for the Context API vs a state management library like Redux or Zustand? 53. What are the core building blocks of Redux (store, actions, reducers)? How does data flow in Redux? 54. What is Redux Toolkit, and why is it now the recommended way to use Redux over the "classic" approach? 55. What is the difference between local component state, global app state, and server state (e.g., React Query/TanStack Query)? 56. Why is it considered bad practice to store server data (like API responses) directly in Redux without a caching strategy? 57. How would you structure state for a real-world e-commerce cart (add, remove, update quantity, persist on refresh)? #### Real-World Application Patterns 58. How would you implement client-side routing in a React app (React Router)? What's the difference between `<Link>` and a normal `<a>` tag? 59. How do you handle protected/private routes in a React app (e.g., redirect unauthenticated users to `/login`)? 60. How would you implement pagination or infinite scroll for a real-world product listing page? 61. How would you handle API calls and loading/error states cleanly in a component (or with React Query)? 62. What is a Higher-Order Component (HOC)? Give a real-world example (e.g., `withAuth`). 63. How do Higher-Order Components compare to custom hooks for sharing logic? Which is generally preferred today and why? 64. What is the Render Props pattern? How does it compare to Hooks for solving the same problem? 65. How would you debounce a search input in a real-world search bar to avoid firing an API call on every keystroke? 66. How would you implement a reusable Modal/Dialog component that can be used across the app? 67. How would you handle global error boundaries in a React app? What is an Error Boundary, and what can/can't it catch? #### Testing 68. What tools are commonly used to test React components (Jest, React Testing Library)? 69. What is the philosophy behind React Testing Library ("test like a user") vs Enzyme's implementation-detail testing? 70. How would you test a component that makes an API call (mocking fetch/axios)? --- ## SECTION 3: ADVANCED LEVEL (Q71–Q100) *Internals, architecture, scalability, system design thinking* #### React Internals 71. What is the Reconciliation algorithm, and what heuristics does React use to make diffing efficient ($O(n)$ instead of $O(n^3)$)? 72. What is Fiber (React Fiber)? What problem did it solve compared to the old stack reconciler? 73. What is the difference between the "render phase" and the "commit phase" in React's rendering process? 74. What is concurrent rendering in React 18? How does it differ from the legacy synchronous rendering model? 75. What are Transitions (`useTransition`, `startTransition`) in React 18, and what real-world problem do they solve (e.g., keeping UI responsive during a large state update)? 76. What is `useDeferredValue`, and how does it differ from debouncing in a real-world search/filter scenario? 77. What is automatic batching in React 18, and how does it differ from React 17's batching behavior? 78. What are Server Components (React Server Components)? How do they differ from traditional Client Components? 79. What is hydration in the context of Server-Side Rendering (SSR)? What can go wrong (hydration mismatch errors)? 80. What is the Suspense boundary used for beyond code-splitting (e.g., data fetching, streaming SSR)? #### Architecture & Real-World System Design 81. How

would you architect a large-scale React application's folder/module structure (feature-based vs type-based)? 82. How would you design a design-system/component library used across multiple teams' React apps? 83. How would you handle internationalization (i18n) in a real-world multi-language React application? 84. How would you implement role-based access control (RBAC) in the UI (hiding/disabling features based on user permissions)? 85. How would you handle real-time data updates in a React app (WebSockets) — e.g., a live order-tracking dashboard? 86. How would you design a form system that handles deeply nested, dynamic fields (e.g., a multi-step checkout or dynamic survey builder)? 87. What is optimistic UI updating? Give a real-world example (e.g., "like" button) and explain how you'd roll back on failure. 88. How would you handle caching, request deduplication, and background refetching for server data in a real-world dashboard (React Query/SWR concepts)? 89. How would you architect a micro-frontend setup where multiple teams ship independent React apps into one shell application? 90. How would you structure a monorepo containing multiple React apps and shared component packages (e.g., using Turborepo/Nx)? ### SSR, Next.js & Build Concepts 91. What is the difference between Client-Side Rendering (CSR), Server-Side Rendering (SSR), and Static Site Generation (SSG)? When would you choose each for a real-world project? 92. What is Incremental Static Regeneration (ISR) in Next.js, and what real-world problem does it solve (e.g., a product page that needs to stay fresh without full rebuilds)? 93. How does Next.js's file-based routing and data-fetching model (e.g., `getServerSideProps`, or the App Router's server components) change how you architect a React app? 94. What is tree-shaking, and how does your code (e.g., barrel imports) affect final bundle size? 95. How would you analyze and reduce a React app's bundle size in a real production project? ### Security & Production Concerns 96. What are common security risks in a React app (XSS via `dangerouslySetInnerHTML`, exposing secrets in client bundles)? 97. Why should sensitive logic (e.g., pricing calculations, permission checks) never be trusted purely on the client side? 98. How would you handle authentication token storage (localStorage vs httpOnly cookies) and what are the security trade-offs of each? 99. How would you set up monitoring/error tracking (e.g., Sentry) in a production React app, and what would you want visibility into? 100. If you were building a real-world analytics dashboard (like a mini Google Analytics) with live-updating charts, filters, thousands of data rows, and role-based views — how would you architect the React app for performance, maintainability, and scalability?

Response: **SECTION 1: BASIC LEVEL (Q1–Q35)**

1. What is React? Why is it called a "library" and not a "framework"?

- **Answer:** React is an open-source, declarative front-end JavaScript library developed by Meta for building user interfaces. It is a *library* because it focuses solely on the view layer (rendering UI) and lets developers choose external tools for routing, state management, and build setups. A *framework* (like Angular) is opinionated and provides an entire architecture out-of-the-box.
- **Real-World Example:** Building a house using React is like buying furniture from a modular catalog (you pick what you want), whereas a framework is like buying a pre-built prefab house where all rooms and plumbing are strictly predetermined.

2. What is the Virtual DOM? Why does React use it instead of directly manipulating the real DOM?

- **Answer:** The Virtual DOM is a lightweight, in-memory JavaScript representation of the actual browser DOM. React uses it because direct manipulation of the real DOM triggers expensive browser layouts, reflows, and repaints, which degrade performance.

- **Real-World Example:** An architect updates a blueprint in pencil first to check measurements before tearing down actual walls on a construction site.

3. How does the Virtual DOM improve performance? Explain the diffing/reconciliation process at a high level.

- **Answer:** When state changes, React creates a new Virtual DOM tree, compares it with the previous tree using a fast $O(n)$ heuristic diffing algorithm, and patches *only* the modified elements into the real DOM.
- **Real-World Example:** Checking a spreadsheet for modified cells and updating only those specific spreadsheet cells instead of re-printing the entire multi-page report.

4. What is JSX? Why isn't it valid JavaScript on its own, and how does it get converted?

- **Answer:** JSX (JavaScript XML) is a syntax extension allowing HTML-like code inside JavaScript. Browsers cannot read it natively; tools like Babel compile it into standard `React.createElement()` calls.
- **Real-World Example:** Translating a human-readable foreign language phrase into machine-executable computer instructions via a compiler.

5. Can you write React without JSX? Give an example using `React.createElement`.

- **Answer:** Yes. JSX is purely syntactic sugar for function calls.
- **Code Example:**

```
const element = React.createElement('h1', { id: 'title' }, 'Hello World');
```

6. What is a React component? What is the difference between a functional and a class component?

- **Answer:** Components are independent, reusable UI building blocks. Functional components are modern JavaScript functions that return JSX and use Hooks; class components are ES6 classes extending `React.Component` that use lifecycle methods and `this.state`.
- **Real-World Example:** A Lego brick that can be reused across different castle designs.

7. What are props? Give a real-world example (e.g., passing user data to a `ProfileCard` component).

- **Answer:** Props (properties) are read-only inputs passed from a parent component down to a child component to configure its behavior or display data.
- **Real-World Example:** Passing user credentials (`name="Alice", role="Admin"`) to an identity badge template.

8. Are props mutable or immutable? What happens if a child tries to modify its own props?

- **Answer:** Props are strictly immutable (read-only) for the child component. If a child attempts to modify them, React violates unidirectional data flow and throws runtime errors or unexpected behavior.

- **Real-World Example:** A tenant reading a lease agreement cannot rewrite the landlord's pricing rules directly on the contract sheet.

9. What is state in React? How is it different from props?

- **Answer:** State is internal, mutable data managed *within* a component that triggers re-renders when updated. Props are external, read-only data passed down from a parent.
- **Real-World Example:** The internal volume level of a TV is state (controlled by the remote), while the manufacturer's user manual specifications are props.

10. What is the difference between controlled and uncontrolled components? Give a form example.

- **Answer:** Controlled components store form input values in React state (`useState`), while uncontrolled components store form values directly in the DOM and access them via `useRef`.
- **Real-World Example:** A live auto-calculating digital shopping checkout form (controlled) vs. a traditional drop-box paper form read only upon hitting submit (uncontrolled).

11. What triggers a component to re-render in React?

- **Answer:** A component re-renders when its internal state changes via setter functions, when its parent component re-renders (passing new props), or when its context values change.
- **Real-World Example:** Refreshing an active live scoreboard widget whenever a new goal is scored.

12. What are keys in React lists, and why does React warn you when they're missing?

- **Answer:** Keys are unique string identifiers (`key={item.id}`) assigned to elements in an array. They help React identify which items have changed, been added, or been removed during reconciliation.
- **Real-World Example:** Roll-call attendance numbers assigned to students so the teacher tracks who left the classroom without misidentifying others.

13. Why shouldn't you use array index as a key when the list can reorder or change? What real bug can this cause?

- **Answer:** Using indexes causes performance issues and UI bugs because item positions change relative to indexes, leading React to mismatch component state (e.g., ticking checkboxes or typing into input fields attaching to the wrong list item).
- **Real-World Example:** Swapping rows in a spreadsheet where line items keep the row number's old data inputs rather than moving with the data row.

14. What is component composition? Give a real-world example (e.g., a Layout component wrapping Header, Sidebar, Content).

- **Answer:** Component composition is the practice of building complex UIs by nesting smaller, specialized components inside container components using the `children` prop.

- **Real-World Example:** Assembling an entire computer system by slotting independent modules (GPU, RAM, CPU) into a motherboard container layout.

15. What is conditional rendering? Show a couple of common patterns (ternary, &&, early return).

- **Answer:** Conditional rendering displays different UI elements based on changing conditions (e.g., authentication status, loading states).
- **Code Example:** `{isLoggedIn ? <Dashboard/> : <Login/>}`, `{hasError && <ErrorMsg/>}`, or returning `null` early if loading.

16. What are React Hooks? Why were they introduced (what problem did they solve)?

- **Answer:** Hooks are functions (`useState`, `useEffect`, etc.) that let functional components manage state and lifecycle methods without writing ES6 classes, solving code reuse and class complexity issues.
- **Real-World Example:** Upgrading a mechanical wristwatch with digital internal microchips to add alarms and stopwatches easily.

17. What is `useState`? Give a real-world example (e.g., toggling a modal open/closed).

- **Answer:** A Hook that lets functional components hook into local state variables and provide a setter function to update them.
- **Code Example:** `const [isOpen, setIsOpen] = useState(false);`

18. What is `useEffect`? What real-world tasks would you put inside it (e.g., fetching data, subscribing to events)?

- **Answer:** A Hook used for handling side effects in functional components, running operations after render cycles.
- **Real-World Example:** Fetching user profile data from a remote REST API right after a dashboard component mounts.

19. What is the dependency array in `useEffect`? What happens if you omit it vs pass an empty array `[]`?

- **Answer:** The dependency array controls when the effect runs. Omitting it runs the effect on every render; passing an empty array `[]` runs it *only once* on mount.
- **Real-World Example:** Setting up a smoke detector alarm that triggers every time room air changes (no array) vs. initializing a room security camera once when the store opens (`[]`).

20. What is the cleanup function in `useEffect`, and when would you need one (e.g., clearing an interval or unsubscribing)?

- **Answer:** A function returned inside `useEffect` that runs right before the component unmounts or before re-running the effect, used to prevent memory leaks.
- **Code Example:** Clearing a `setInterval` or removing a window resize event listener.

21. How does event handling in React differ from vanilla JavaScript (SyntheticEvent)?

- **Answer:** React uses cross-browser wrapper objects called `SyntheticEvent` that normalize browser behaviors so events work identically across all browsers.
- **Real-World Example:** A universal car steering wheel adapter that fits onto various car model columns seamlessly.

22. How do you handle a form input's value with `useState`? What is a "controlled input"?

- **Answer:** By binding the input's `value` attribute to state and updating it via an `onChange` event handler.
- **Real-World Example:** A digital savings account balance field that updates instantly as you type deposit amounts.

23. How do you handle multiple form fields with a single state object instead of many `useState` calls?

- **Answer:** By using a single state object and updating dynamic keys using computed property names: `setForm({ ...form, [e.target.name]: e.target.value })`.
- **Real-World Example:** Filling out a multi-field user registration form object rather than storing separate variables for first name, last name, and email.

24. What is `preventDefault()` used for in a React form submit handler, and why is it usually necessary?

- **Answer:** It stops the browser's default page reload behavior on form submission, allowing React to process form data via JavaScript asynchronously (Single-Page Application behavior).
- **Real-World Example:** Stopping a browser from refreshing a page and wiping out unsaved data when you click submit.

25. How would you validate a login form in React before submitting to an API?

- **Answer:** By checking input state values against validation rules (e.g., checking email format or minimum password length) and storing error messages in local state before triggering an API call.
- **Real-World Example:** A security checkpoint gate that flashes red if your ticket barcode is incomplete before letting you walk through.

26. What are the different ways to style a React component (CSS files, CSS Modules, inline styles, styled-components)?

- **Answer:** Standard CSS stylesheets, scoped CSS Modules (`*.module.css`), inline style objects (`style={{ color: 'red' }}`), and CSS-in-JS libraries (styled-components).
- **Real-World Example:** Painting rooms using a global building paint color code vs. individualized room designer wallpaper labels.

27. What is the `className` prop, and why isn't it called `class` like in HTML?

- **Answer:** It is named `className` because `class` is a reserved keyword in JavaScript (used for defining ES6 classes).
- **Real-World Example:** Naming an internal variable `className` to avoid crashing JavaScript parser execution.

28. What is `React.Fragment (<>...</>)` used for, and why avoid unnecessary wrapper `<div>s`?

- **Answer:** Fragments allow grouping multiple JSX children without adding extra DOM nodes to the HTML tree, keeping CSS layouts (like CSS Grid/Flexbox) clean and unbroken.
- **Real-World Example:** Packing items inside a cardboard box without adding extra unnecessary filler boxes inside.

29. What is the `children` prop? Give a real-world example (e.g., a reusable `Card` or `Modal` wrapper component).

- **Answer:** A special prop that automatically passes whatever elements are nested inside a component's opening and closing tags.
- **Code Example:** `<Card><h3>Title</h3><p>Body text</p></Card>` where `Card` renders `props.children`.

30. What is prop drilling? Give a real-world scenario where it becomes a problem.

- **Answer:** Passing props down through multiple intermediate component layers just to reach a deeply nested child component that actually needs the data.
- **Real-World Example:** Handing a package down a human bucket brigade line of 10 people when only the last person needs it.

31. What is `create-react-app` / `Vite`, and what's the difference between them for starting a new project?

- **Answer:** Build tools and bundlers used to scaffold React apps. Vite uses native ES modules during development, making server startup and Hot Module Replacement (HMR) significantly faster than `create-react-app` (which uses Webpack).
- **Real-World Example:** Upgrading from an old sluggish delivery truck to a high-speed electric cargo vehicle.

32. What is the role of the `public/index.html` file and the root `div` in a React app?

- **Answer:** It acts as the single entry HTML file where React injects the compiled JavaScript application (`<div id="root"></div>`).
- **Real-World Example:** The main stage arena platform where an entire theatrical performance takes place.

33. What is `ReactDOM.createRoot()` (React 18) used for?

- **Answer:** It creates a root container for a React application, enabling concurrent rendering features in React 18 and replacing the legacy `ReactDOM.render()`.
- **Real-World Example:** Setting up the main power grid breaker switch for an entire building block.

34. How do you pass a function from a parent to a child component to handle a child-triggered event?

- **Answer:** The parent passes a callback function down as a prop, which the child invokes when an event (like a button click) occurs.
- **Real-World Example:** A parent giving a walkie-talkie to a child so the child can call back when dinner is ready.

35. What is the difference between a "presentational" and a "container" component in real-world app structuring?

- **Answer:** Presentational components focus on *how things look* (stateless UI markup), while container components focus on *how things work* (fetching data, managing state, business logic).
- **Real-World Example:** A TV monitor screen (presentational) versus the internal circuit motherboard processor (container).

SECTION 2: INTERMEDIATE LEVEL (Q36–Q70)

36. What is `useContext`, and how does it help avoid prop drilling? Give a real-world example (e.g., theme or auth context).

- **Answer:** `useContext` allows components to consume data directly from a global context provider tree without passing props through every intermediate level.
- **Real-World Example:** A building intercom paging system broadcasting public announcements to all offices simultaneously without individual memos.

37. What is `useRef`? Give two real-world use cases (DOM access, and persisting a mutable value without re-render).

- **Answer:** A Hook that returns a mutable object whose `.current` property persists across renders without triggering re-renders when changed.
- **Real-World Example:** Focusing a text input box programmatically or storing a timer interval ID reference.

38. What is the difference between `useRef` and `useState` in terms of triggering re-renders?

- **Answer:** Updating `useState` triggers a component re-render, whereas modifying `useRef.current` does not trigger re-renders.
- **Real-World Example:** A scoreboard display screen update (`useState`) versus a background counter log notepad (`useRef`).

39. What is `useReducer`? When would you prefer it over `useState` (e.g., complex form or shopping cart state)?

- **Answer:** A Hook alternative to `useState` for managing complex state logic involving multiple sub-values or predictable state transitions handled via actions and a reducer function.
- **Real-World Example:** Managing a shopping cart state (adding, removing, clearing items) with distinct action types.

40. What are custom hooks? Write/describe a real-world custom hook (e.g., `useFetch`, `useDebounce`, `useLocalStorage`).

- **Answer:** JavaScript functions whose names start with "use" that extract reusable stateful logic into modular helper functions.
- **Real-World Example:** A custom `useLocalStorage` hook to sync form state with browser local storage automatically.

41. What rules must you follow when using Hooks (Rules of Hooks), and why do they exist?

- **Answer:** Call hooks only at the top level of your React function (never inside loops, conditions, or nested functions) to ensure React maintains correct state order across renders.
- **Real-World Example:** Following strict seating chart rules in an auditorium so attendance sheets match individuals accurately.

42. What is `useMemo`, and when should you actually use it? Give a real-world case (e.g., expensive filtering/sorting of a large list).

- **Answer:** A Hook that caches the result of an expensive calculation between renders unless its dependencies change.
- **Real-World Example:** Caching the filtered search results of a large product inventory catalog to avoid recalculating on every keystroke.

43. What is `useCallback`, and how does it differ from `useMemo`?

- **Answer:** `useCallback` caches and memoizes a *function definition* across renders, whereas `useMemo` caches the *return value* of a calculation.
- **Real-World Example:** Re-using a stable function reference passed down to child list items to prevent child re-renders.

44. Why can overusing `useMemo`/`useCallback` sometimes hurt performance instead of helping?

- **Answer:** They add computational overhead (memory allocation and dependency array checks on every render) which can slow down apps if applied unnecessarily to lightweight calculations.
- **Real-World Example:** Hiring a full-time accountant just to calculate simple pocket change math.

45. What is the difference between `useLayoutEffect` and `useEffect`? Give a real-world case where `useLayoutEffect` is necessary.

- **Answer:** `useLayoutEffect` fires synchronously *before* the browser paints the screen, whereas `useEffect` fires *after* paint. Use `useLayoutEffect` to prevent visual flicker when measuring DOM element sizes.
- **Real-World Example:** Calculating tooltip popup placement coordinates before the user sees the element flash into view.

46. What is `React.memo`, and when would you use it? Give a real-world example (e.g., a large list item component).

- **Answer:** A higher-order component that memoizes a functional component, skipping re-renders if its props haven't changed.
- **Real-World Example:** Optimizing a large table row component so unmodified rows don't re-render when another row updates.

47. Why doesn't `React.memo` help if you're passing a new function or object as a prop on every render?

- **Answer:** Because shallow prop comparison sees new inline object/function references (`{}` or `() => {}`) as different on every render, defeating the memoization cache.
- **Real-World Example:** Handing out fresh duplicate keys every single morning instead of keeping the same master key chain.

48. What is code-splitting in React? How does `React.lazy` and `Suspense` help with real-world bundle size issues?

- **Answer:** Splitting your app bundle into smaller chunks that load on-demand, reducing initial page load times using `React.lazy` and fallback UI loaders (`Suspense`).
- **Real-World Example:** Loading chapter books page-by-page on a tablet reader rather than downloading an entire encyclopedia at once.

49. How would you optimize the rendering of a list with thousands of items (virtualization, e.g., `react-window`)?

- **Answer:** By rendering only the items currently visible in the user's viewport window rather than rendering all 10,000 DOM nodes at once.
- **Real-World Example:** Showing only the visible table rows on your computer screen while scrolling through a massive database log.

50. What causes unnecessary re-renders in a React app, and how would you debug them (React DevTools Profiler)?

- **Answer:** Unnecessary re-renders are caused by parent state updates cascading down to child components, unstable prop references, or broad context updates. Debug using React DevTools Profiler to record render times and highlighted updates.

- **Real-World Example:** Using thermal imaging cameras to track heat leaks and inefficient energy loss in a house.

51. What is "lifting state up"? Give a real-world example (e.g., two sibling components needing shared filter state).

- **Answer:** Moving state from child components up to their closest common parent component so sibling components can share and synchronize that state.
- **Real-World Example:** Two different thermostats in adjacent rooms wired to a central furnace controller unit.

52. When would you reach for the Context API vs a state management library like Redux or Zustand?

- **Answer:** Use Context for low-frequency global data (themes, auth status, locale); use Redux/Zustand for complex global state with frequent updates, middleware, and advanced debugging requirements.
- **Real-World Example:** Using a building intercom for public announcements (Context) vs. a complex corporate inventory supply chain system (Redux).

53. What are the core building blocks of Redux (store, actions, reducers)? How does data flow in Redux?

- **Answer:** Store (holds app state), Actions (describe what happened), and Reducers (pure functions specifying how state changes based on actions). Data flows in a strict unidirectional cycle: Action dispatched → Reducer updates Store → UI re-renders.
- **Real-World Example:** A bank ledger transaction system (Action: deposit, Reducer: updates account balance store).

54. What is Redux Toolkit, and why is it now the recommended way to use Redux over the "classic" approach?

- **Answer:** Redux Toolkit (RTK) is the official opinionated toolset for Redux that eliminates boilerplate code, configures store automatically, and includes built-in immer support for immutable state updates.
- **Real-World Example:** Upgrading from manual gear-shift tractors to modern automated transmission vehicles.

55. What is the difference between local component state, global app state, and server state (e.g., React Query/TanStack Query)?

- **Answer:** Local state belongs to one component; global state is shared across multiple UI components; server state originates remotely from a database and requires caching, background refetching, and synchronization.
- **Real-World Example:** A user's open modal form toggle (local), shopping cart items (global), and remote product inventory stock data from a database (server).

56. Why is it considered bad practice to store server data (like API responses) directly in Redux without a caching strategy?

- **Answer:** Because server data becomes stale quickly, requiring complex manual logic for background syncing, garbage collection, error handling, and race conditions that dedicated server-state libraries handle automatically.
- **Real-World Example:** Storing printed weather forecasts in a vault without checking live satellite updates throughout the day.

57. How would you structure state for a real-world e-commerce cart (add, remove, update quantity, persist on refresh)?

- **Answer:** Maintain an array of cart items in state/global store with item IDs and quantities, and sync updates to `localStorage` or a backend database via effects or actions.
- **Real-World Example:** A physical shopping trolley where items are added, quantities adjusted, and saved at the checkout counter.

58. How would you implement client-side routing in a React app (React Router)? What's the difference between `<Link>` and a normal `<a>` tag?

- **Answer:** React Router manages views client-side without full page reloads. `<Link>` intercepts clicks to update the URL and render components via history API, whereas an `<a>` tag triggers a full browser reload.
- **Real-World Example:** Switching tabs inside a single-page web app dashboard vs. opening an entirely new website URL.

59. How do you handle protected/private routes in a React app (e.g., redirect unauthenticated users to `/login`)?

- **Answer:** By creating a wrapper component that checks user authentication state and either renders the requested protected component or redirects using `<Navigate>` / `useNavigate`.
- **Real-World Example:** A VIP lounge security door guard checking membership badges before letting guests enter.

60. How would you implement pagination or infinite scroll for a real-world product listing page?

- **Answer:** Fetch data in chunks using API query parameters (`page`, `limit`) triggered by button clicks (pagination) or by intersection observer scroll tracking (infinite scroll).
- **Real-World Example:** Flipping pages in a catalog book vs. scrolling down an endless social media photo feed.

61. How would you handle API calls and loading/error states cleanly in a component (or with React Query)?

- **Answer:** Track `isLoading`, `isError`, and `data` states using React Query hooks or local `useState` combined with `try/catch/finally` blocks.

- **Real-World Example:** Displaying a loading spinner while fetching data, an error banner if the network fails, and the data grid when successful.

62. What is a Higher-Order Component (HOC)? Give a real-world example (e.g., `withAuth`).

- **Answer:** A function that takes a component and returns a new enhanced component with added behavior or props.
- **Code Example:** `const ProtectedDashboard = withAuth(Dashboard);`

63. How do Higher-Order Components compare to custom hooks for sharing logic? Which is generally preferred today and why?

- **Answer:** Custom hooks are preferred today over HOCs because they avoid component tree nesting clutter ("wrapper hell") and make sharing stateful logic cleaner and more readable.
- **Real-World Example:** Upgrading from wearing multiple heavy winter jacket layers (HOCs) to using a smart temperature-regulated vest (Hooks).

64. What is the Render Props pattern? How does it compare to Hooks for solving the same problem?

- **Answer:** A technique where a component takes a function prop that returns a React element to share code. Hooks have largely superseded render props due to cleaner syntax without nested callbacks.
- **Real-World Example:** Passing a rendering blueprint function callback to a generic data-fetching container wrapper.

65. How would you debounce a search input in a real-world search bar to avoid firing an API call on every keystroke?

- **Answer:** Use a timer (`setTimeout`) combined with `useEffect` or utility libraries like `Lodash` to delay API search requests until the user stops typing for a set duration (e.g., 300ms).
- **Real-World Example:** A smart elevator that waits until you finish speaking your destination floor before moving.

66. How would you implement a reusable Modal/Dialog component that can be used across the app?

- **Answer:** Create a modal component using React Portals (`ReactDOM.createPortal`) to render outside the normal DOM hierarchy into a root modal container, driven by `open/close` props.
- **Real-World Example:** A pop-up dialog alert window layered directly on top of your browser workspace.

67. How would you handle global error boundaries in a React app? What is an Error Boundary, and what can/can't it catch?

- **Answer:** An Error Boundary is a class component implementing `componentDidCatch` or `getDerivedStateFromError` that catches JavaScript errors anywhere in its child component

tree. It *cannot* catch errors in event handlers, asynchronous code, or server-side rendering.

- **Real-World Example:** An emergency circuit breaker fuse box that stops electrical surges from destroying an entire appliance system.

68. What tools are commonly used to test React components (Jest, React Testing Library)?

- **Answer:** Jest serves as the test runner and assertion library, while React Testing Library (RTL) provides utilities to test components from a user-centric perspective.
- **Real-World Example:** Automated factory quality control testing machines checking product buttons and displays.

69. What is the philosophy behind React Testing Library ("test like a user") vs Enzyme's implementation-detail testing?

- **Answer:** RTL focuses on testing what the user sees and interacts with (DOM nodes, text, roles) rather than internal component states, methods, or prop structures (Enzyme), leading to more resilient tests.
- **Real-World Example:** Testing a car by sitting in the driver's seat and turning the steering wheel (RTL) vs. taking apart the dashboard wiring to check individual resistors (Enzyme).

70. How would you test a component that makes an API call (mocking fetch/axios)?

- **Answer:** By using mocking libraries (like Jest mocks or MSW - Mock Service Worker) to intercept network requests and return simulated mock data responses during tests.
- **Real-World Example:** Using a driving simulator crash-test dummy setup rather than driving a real car into a brick wall.

SECTION 3: ADVANCED LEVEL (Q71–Q100)

71. What is the Reconciliation algorithm, and what heuristics does React use to make diffing efficient ($O(n)$ instead of $O(n^3)$)?

- **Answer:** React's heuristic diffing algorithm assumes two elements of different types produce different trees, and stable key props identify child elements across renders, reducing tree comparison complexity from $O(n^3)$ to $O(n)$.
- **Real-World Example:** Comparing two inventory audit sheets by scanning barcode IDs rather than cross-comparing every physical item individually.

72. What is Fiber (React Fiber)? What problem did it solve compared to the old stack reconciler?

- **Answer:** React Fiber is a reimplementation of React's core reconciliation engine that enables incremental rendering, work prioritization, pausing, and resuming rendering tasks.
- **Real-World Example:** A modern multi-tasking operating system processor that pauses background file downloads to render mouse cursor movements smoothly.

73. What is the difference between the "render phase" and the "commit phase" in React's rendering process?

- **Answer:** The render phase calculates UI changes by calling components and running diffing (can be paused or aborted). The commit phase applies those changes to the real DOM and runs side effects (cannot be interrupted).
- **Real-World Example:** An architect drawing blueprinted revisions in pencil (render) versus construction crews pouring concrete foundation walls (commit).

74. What is concurrent rendering in React 18? How does it differ from the legacy synchronous rendering model?

- **Answer:** Concurrent rendering allows React to prepare multiple versions of the UI simultaneously, interrupting and resuming rendering work based on user interaction priority.
- **Real-World Example:** A high-speed web browser engine downloading media streams in chunks while keeping user scrolling animations buttery smooth.

75. What are Transitions (`useTransition`, `startTransition`) in React 18, and what real-world problem do they solve?

- **Answer:** Hooks that mark state updates as non-urgent transitions, letting urgent updates (like typing or clicks) interrupt heavy background renders to keep the UI responsive.
- **Real-World Example:** Typing into a search box instantly while heavy search result filters compute in the background without freezing the keyboard.

76. What is `useDeferredValue`, and how does it differ from debouncing in a real-world search/filter scenario?

- **Answer:** `useDeferredValue` lets you defer re-rendering a non-urgent part of the UI until urgent updates finish, adapting automatically to user device performance unlike fixed-timer debouncing.
- **Real-World Example:** A camera lens auto-focusing on the main subject first before sharpening background scenery details.

77. What is automatic batching in React 18, and how does it differ from React 17's batching behavior?

- **Answer:** React 18 automatically batches multiple state updates into a single re-render across timeouts, promises, and native event handlers, whereas React 17 batched updates only inside React event handlers.
- **Real-World Example:** Bundling multiple separate shipping parcels into a single delivery truck trip instead of dispatching trucks for every single item.

78. What are Server Components (React Server Components)? How do they differ from traditional Client Components?

- **Answer:** Server Components render entirely on the server, sending zero JavaScript bundle weight to the client, while Client Components execute on the browser and handle interactivity.

- **Real-World Example:** A server-side restaurant menu printout (Server Component) versus an interactive table ordering touchscreen tablet (Client Component).

79. What is hydration in the context of Server-Side Rendering (SSR)? What can go wrong (hydration mismatch errors)?

- **Answer:** Hydration is the process where client-side React attaches event listeners to pre-rendered HTML received from the server. Mismatch errors occur when server and client initial HTML output differ.
- **Real-World Example:** Trying to plug pre-built wall electrical sockets into mismatched wiring harnesses during final assembly.

80. What is the Suspense boundary used for beyond code-splitting (e.g., data fetching, streaming SSR)?

- **Answer:** Suspense lets components pause rendering while waiting for asynchronous data or code chunks to load, displaying fallback UI loading states in the interim.
- **Real-World Example:** Displaying a skeleton placeholder loading screen on a video streaming app while video data buffers.

81. How would you architect a large-scale React application's folder/module structure (feature-based vs type-based)?

- **Answer:** Use a feature-based folder structure (grouping components, hooks, and services by business feature like `auth/`, `dashboard/`) rather than a type-based structure (`components/`, `hooks/`) to improve scalability and maintainability.
- **Real-World Example:** Organizing a massive library archive by distinct genre departments rather than dumping all books into one giant pile.

82. How would you design a design-system/component library used across multiple teams' React apps?

- **Answer:** Build an isolated, accessible component library packaged with Storybook for documentation, styled with design tokens, and published via a private npm registry.
- **Real-World Example:** Standardized highway road signs and traffic lights manufactured according to universal state regulations.

83. How would you handle internationalization (i18n) in a real-world multi-language React application?

- **Answer:** Using libraries like `react-i18next` to manage translated message string dictionaries, dynamic locale switching, and number/date formatting.
- **Real-World Example:** Installing automatic audio translation headsets for international delegates at a global summit conference.

84. How would you implement role-based access control (RBAC) in the UI (hiding/disabling features based on user permissions)?

- **Answer:** By wrapping UI components or routes with authorization wrapper checks that verify user permission roles against stored user tokens.
- **Real-World Example:** Bank teller station computer terminals locking manager-only administrative override buttons unless authorized.

85. How would you handle real-time data updates in a React app (WebSockets) — e.g., a live order-tracking dashboard?

- **Answer:** Establishing a persistent WebSocket connection inside a `useEffect` hook to stream live server push events directly into component state.
- **Real-World Example:** A live stock market ticker screen updating prices instantly as trades occur globally.

86. How would you design a form system that handles deeply nested, dynamic fields (e.g., a multi-step checkout or survey builder)?

- **Answer:** Using form state management libraries like React Hook Form combined with schema validation libraries like Zod or Yup to handle complex nested arrays and validation schemas.
- **Real-World Example:** A modular dynamic tax return form that adds or removes dependent sections based on your previous answers.

87. What is optimistic UI updating? Give a real-world example (e.g., "like" button) and explain how you'd roll back on failure.

- **Answer:** Updating the UI immediately assuming an API request will succeed, and rolling back the state change with an error message if the server request fails.
- **Real-World Example:** Liking a social media post where the heart icon turns red instantly before the network confirmation packet even reaches the server.

88. How would you handle caching, request deduplication, and background refetching for server data in a real-world dashboard?

- **Answer:** By using modern data-fetching libraries like TanStack Query (React Query) or SWR, which manage request caches, deduplication keys, and automatic stale-while-revalidate background fetches.
- **Real-World Example:** A weather station app displaying cached forecast data instantly while checking for updated satellite weather data quietly in the background.

89. How would you architect a micro-frontend setup where multiple teams ship independent React apps into one shell application?

- **Answer:** Using Module Federation (Webpack 5) or single-spa to load independent React micro-app bundles dynamically into a shared host shell container application.
- **Real-World Example:** A massive shopping mall housing independent retail stores operated by separate companies under one shared roof structure.

90. How would you structure a monorepo containing multiple React apps and shared component packages (e.g., using Turborepo/Nx)?

- **Answer:** Using monorepo build tools like Turborepo or Nx to manage workspace packages, shared UI components, and incremental task caching across multiple apps.
- **Real-World Example:** A corporate engineering headquarters managing multiple vehicle assembly lines sharing common engine component warehouses.

91. What is the difference between Client-Side Rendering (CSR), Server-Side Rendering (SSR), and Static Site Generation (SSG)?

- **Answer:** CSR renders pages in the browser via JavaScript; SSR generates HTML on the server on every request; SSG generates static HTML pages at build time.
- **Real-World Example:** A dynamic single-page web app dashboard (CSR), a live news article publishing portal (SSR), and a pre-printed static company marketing brochure website (SSG).

92. What is Incremental Static Regeneration (ISR) in Next.js, and what real-world problem does it solve?

- **Answer:** ISR allows you to update static pages in the background incrementally after build time without rebuilding your entire website.
- **Real-World Example:** Updating a daily newspaper front-page headline online without having to reprint the entire edition from scratch.

93. How does Next.js's file-based routing and data-fetching model change how you architect a React app?

- **Answer:** File-based routing automatically maps folder directories to URL paths, and Server Components fetch data directly inside components without boilerplate Redux thunks or complex effect hooks.
- **Real-World Example:** Automatically assigning office room numbers based on physical hallway floor maps.

94. What is tree-shaking, and how does your code (e.g., barrel imports) affect final bundle size?

- **Answer:** Tree-shaking is a build optimization process that removes unused dead code from final production bundles. Poor barrel file imports (`import * from ...`) can sometimes prevent effective tree-shaking and bloat bundle sizes.
- **Real-World Example:** Packing luggage by discarding unused clothes rather than bringing an entire overflowing closet on a trip.

95. How would you analyze and reduce a React app's bundle size in a real production project?

- **Answer:** Using bundle analysis tools (like `webpack-bundle-analyzer` or Vite visualizer plugins) to inspect heavy packages, implement code-splitting, and replace bloated libraries with lightweight alternatives.

- **Real-World Example:** Weighing a shipping cargo container and auditing items to remove unnecessary heavy packaging material.

96. What are common security risks in a React app (XSS via dangerouslySetInnerHTML, exposing secrets in client bundles)?

- **Answer:** Cross-Site Scripting (XSS) via unsafe HTML injection, and storing sensitive API secret keys in client-side code bundles where users can inspect them.
- **Real-World Example:** Leaving physical vault combination keys taped to the outside front entrance glass door.

97. Why should sensitive logic (e.g., pricing calculations, permission checks) never be trusted purely on the client side?

- **Answer:** Because client-side code runs in the user's browser, making it trivial for malicious users to inspect, intercept, modify, or bypass JavaScript execution.
- **Real-World Example:** Trusting a customer to write their own restaurant bill totals on a napkin without cashier verification.

98. How would you handle authentication token storage (localStorage vs httpOnly cookies) and what are the security trade-offs?

- **Answer:** httpOnly cookies protect authentication tokens from JavaScript XSS attacks (as they are inaccessible to client scripts), whereas localStorage tokens are vulnerable to XSS but easier to attach to client API requests.
- **Real-World Example:** Storing high-security valuables in a bank safe deposit box (httpOnly) vs. carrying loose cash in an outer jacket pocket (localStorage).

99. How would you set up monitoring/error tracking (e.g., Sentry) in a production React app, and what would you want visibility into?

- **Answer:** Integrate error boundary wrappers and SDKs like Sentry to capture unhandled runtime exceptions, stack traces, user session breadcrumbs, and performance metrics.
- **Real-World Example:** Installing an airplane black-box flight recorder to diagnose mechanical failure causes during flights.

100. If you were building a real-world analytics dashboard with live-updating charts, filters, thousands of data rows, and role-based views — how would you architect it for performance and scalability?

- **Answer:** Architect the system using feature-based modular folder structures, virtualization for large data grids (react-window), TanStack Query for server-state caching and background polling, WebSockets for live updates, React 18 concurrent transitions for heavy filters, and Error Boundaries for resilience.
- **Real-World Example:** Building an air traffic control radar command center that manages live flight telemetry streams, user security clearances, and high-frequency data grids smoothly.